Does context help phrase matching?

Tasks 1-3 | Same held-out anchors in every comparison

Protocol and leakage control

Official Kaggle data: 36,473 labelled pairs and 733 distinct anchors. GroupShuffleSplit (seed 42, validation fraction 0.20) gives 28,914 training rows and 7,559 validation rows from 147 held-out anchors. Training and validation anchors are disjoint. Holding out entire anchors prevents the model from exploiting familiarity with an anchor seen during training; a random row split can overstate generalization to new anchors.

The 36 visible test rows are only a format check, contain no labels, and have IDs also found in the official training file. They are never used for model selection or a reported test score. The released full dataset and hidden-test labels were not used.

Baseline and controlled experiments

DeBERTa-v3-small cross-encoder; regression head and MSE; maximum length 64; three epochs; learning rate 2e-5; batch size 32; warmup 0.10; weight decay 0.01; FP16 mixed precision. Each context run starts from the same pretrained checkpoint and seed. Only context text changes. Training time includes epoch validation; peak GPU memory is PyTorch allocated memory.

Model / context	Val. r	Train min	Delta r	Interpretation
Section title (baseline)	0.8178	3.48	+0.0000	Unchanged reference
No context	0.8168	3.30	-0.0010	Similar; may be seed noise
Raw CPC code	0.8195	3.38	+0.0018	Similar; may be seed noise
MiniLM bi-encoder; no training	0.5814	0.00	-	Different architecture and pretraining

MiniLM validation inference: 0.99 s. Peak allocated GPU memory for the three training runs: 2.73 GB, 2.66 GB, 2.71 GB.

Removing context changes r by -0.0010; replacing section titles with raw CPC codes changes it by +0.0018. Both are smaller than the lab's 0.005 noise caution, so neither establishes a reliable improvement. Raw codes are selected by the highest observed validation score, without claiming they are generally better.

The trained baseline exceeds the pretrained bi-encoder by 0.2363 Pearson points on the same rows. This is a substantial observed system-level difference, but the comparison also changes architecture and pretraining. The three context runs together took 10.16 minutes of measured training time.

What the comparison establishes

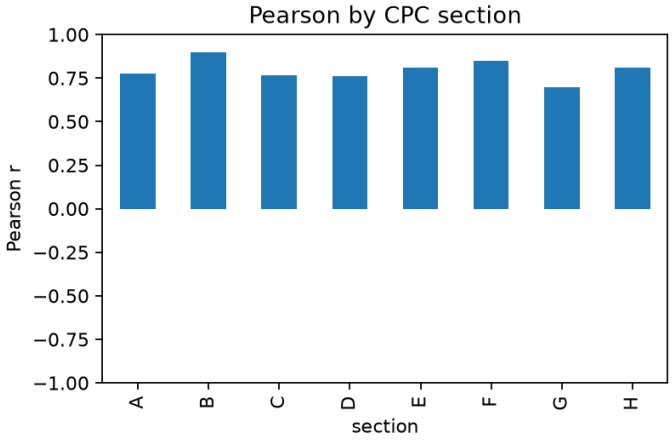
The bi-encoder embeds each phrase independently and compares normalized embeddings by cosine. The cross-encoder allows attention between both phrases and learns from labelled pairs. Their difference combines architecture, checkpoint pretraining, context and task fine-tuning; it is not a causal estimate of fine-tuning alone. Both use exactly the same validation rows.

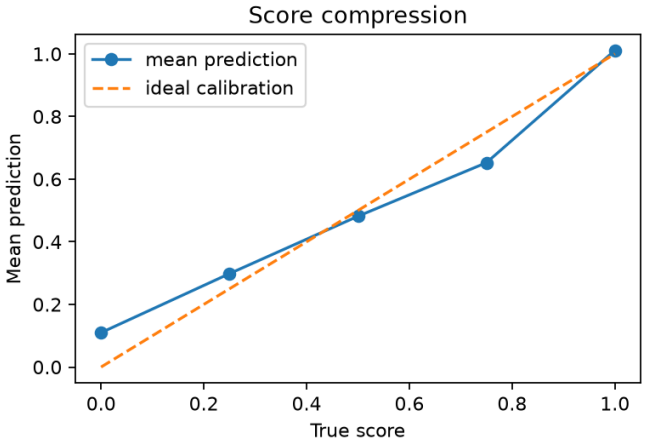
Runtime: RTX 2000 Ada 16 GB; PyTorch 2.10.0+cu128; Transformers 4.57.1. Only local paths and torch_dtype compatibility were adapted from the starter; its baseline settings were retained.

Where the selected model struggles

Task 4 | Counts accompany section metrics and label averages

Selected by validation Pearson: **Raw CPC code**, r = **0.8195**. The same fixed split is used throughout. A single split and seed limit claims about generalization.





CPC	Validation n	Pearson r	True score	n	Mean prediction
A	943	0.7778	0.00	1522	0.110
B	1926	0.8985	0.25	2465	0.298
C	1177	0.7667	0.50	2475	0.482
D	199	0.7608	0.75	832	0.653
E	373	0.8119	1.00	265	1.011
F	968	0.8490			
G	1040	0.6991			
H	933	0.8090			

Section G is hardest here (r = 0.6991, n = 1,040), followed by D (0.7608, n = 199) and C (0.7667, n = 1,177). B is strongest (0.8985, n = 1,926). D has relatively few examples, so its estimate is less stable; section rankings are descriptive rather than evidence of intrinsic difficulty.

G contributes four of the ten largest errors, spanning G02, G06 and G10. They involve phrase structure, related spatial terms and a technical component relation. A plausible explanation is that lexical resemblance is easier to learn than these distinctions from short phrases and a code. This hypothesis needs broader error review; ten extreme cases cannot establish the cause of all section-level differences.

There is partial compression: true 0 averages 0.110 and true 0.75 averages 0.653. Scores 0.25 and 0.5 make up 65.3% of the training data, so an MSE-trained model can favor central predictions when uncertain. But compression is not universal: true 1 averages 1.011, slightly above the valid range. The regression head is unconstrained; clipping is applied only to submission predictions.

Pearson measures linear association, not absolute calibration. A positive rescaling or shift preserves it, while nonlinear transformations can change it. Mean predictions by label therefore provide information that the correlation alone misses.

Ten largest validation errors

Task 4 | Raw predictions; absolute error sorted descending

Failure groups below are an AI-assisted qualitative interpretation of observed validation errors, not new expert annotations. The original labels remain unchanged.

# / group	Anchor -> target (CPC)	True	Pred.	|Error|
1 / S	punch face -> face punch (B05)	0.00	0.971	0.971
2 / S	imaging axis -> imaging of axis (G02)	0.00	0.812	0.812
3 / S	boom hydraulic cylinder -> boom a hydraulic cylinder (E02)	0.00	0.792	0.792
4 / S	imaging axis -> image of axis (G02)	0.00	0.780	0.780
5 / S	return structure -> return to structure (D01)	0.00	0.736	0.736
6 / M	activating position -> active region (G06)	0.00	0.713	0.713
7 / R	breech mechanism -> breech loading naval gun (B29)	0.75	0.043	0.707
8 / M	secure to guard -> security guard (A01)	0.00	0.655	0.655
9 / M	extracting process -> process mining (C13)	0.00	0.649	0.649
10 / R	end pins -> wood pin (G10)	0.75	0.105	0.645

S - Phrase structure (rows 1-5). Shared words overwhelm changes in word order or grammatical role. 'Punch face' names a possible part, while 'face punch' can describe an action. Likewise, an 'imaging axis' differs from imaging an axis, and a 'return structure' differs from returning to a structure. The model assigns high similarity despite zero labels.

M - Meaning substitution (rows 6, 8, 9). 'Activating' and 'active', 'secure' and 'security', or 'extracting' and 'mining' invite lexical associations. Those associations do not establish that the phrases have the same function in a patent. These predictions range from 0.649 to 0.713 despite zero labels.

R - Technical relations (rows 7, 10). The model misses the labelled similarity between a breech mechanism and a breech-loading gun, and between end pins and a wood pin. Component-to-system or role-to-material relationships may require more context than the short input provides. These are the two severe underestimates; the other eight errors are overestimates.

Debatable example: 'boom hydraulic cylinder' versus 'boom a hydraulic cylinder' differs only by an added article and has an awkward construction, making its zero label surprising. Without the original patent sentences or expert rationale, I cannot resolve whether this is a meaningful distinction or annotation ambiguity. I retain the supplied label and report the error rather than correcting it by guesswork.

Reproducibility and submission

Task 5 | Validation selection precedes any hidden-test feedback

Current evidence

Tasks 1-4 use the measured local experiments. The validation-selected raw-CPC configuration also completed an offline Kaggle GPU run and a scored late submission on 12 November 2026. Local validation r = 0.8195; the Kaggle execution gives r = 0.8192. The final notebook is public, and the separate Submissions screenshot records both hidden-test scores.

Validation Pearson	Public score	Private score
0.8192 (Kaggle)	0.7904	0.8102

Public minus Kaggle validation is -0.0288; private minus validation is -0.0090; public minus private is -0.0198. Different anchors, CPC-section mixes and sample sizes can explain gaps. Selecting among three configurations on one validation split can also make its winner optimistic. These are plausible explanations, not verified causes.

Kaggle validation differs from local by -0.0003. The T4/PyTorch 2.11/Transformers 5.20.2 environment differs from the local RTX 2000 Ada/Transformers 4.57.1 setup, so exact numerical replication is not expected. All controlled comparisons in Tasks 1-4 use one local environment.

Final notebook and submission record

The final notebook loads attached weights with internet off and writes submission.csv (id,score) for the current test-row count. Submission predictions are clipped to [0,1], following the starter; validation uses raw predictions. This nonlinear clipping can change Pearson.

One scored competition submission: version 2, script ID 368715520, public 0.7904, private 0.8102. No test score influenced model selection. The first saved Kaggle execution failed before training because Transformers 5.20 removed warmup_ratio; it was not submitted. The successful version uses the equivalent 272 warmup steps (10% of 2,712), one T4, and unchanged model, split, batch size and three-epoch budget.

Final Kaggle notebook version (click to open). Score evidence: kaggle_submissions_VanshJoshi.png, supplied separately.

Limitations and conclusion

The trained cross-encoder performs substantially better than the pretrained sentence-similarity system on held-out anchors. The controlled context changes are effectively tied at this resolution. Main weaknesses are confusing shared wording with equivalent meaning and missing technical relationships. More seeds would be needed to support a context advantage; no optional tuning runs or hidden-label lookups were used.

References and attribution

[1] Course project brief and patent_phrase_starter.ipynb, bk-anupam/generative_ai_shared, module3/labs/patent_phrase_matching. Baseline code adapted from the supplied starter.

[2] Kaggle. U.S. Patent Phrase to Phrase Matching: Data, Rules and code competition. kaggle.com/competitions/us-patent-phrase-to-phrase-matching.

[3] He, Gao & Chen (2021). DeBERTaV3. arXiv:2111.09543. Pretrained checkpoint: huggingface.co/microsoft/deberta-v3-small.

[4] Reimers & Gurevych (2019). Sentence-BERT. arXiv:1908.10084. Checkpoint: huggingface.co/sentence-transformers/all-MiniLM-L6-v2.

[5] Hugging Face Transformers Trainer and Sentence Transformers documentation. huggingface.co/docs/transformers/main_classes/trainer; sbert.net.

Prepared with Codex assistance for code, execution and analysis; student review and understanding are required. No other student solution or public competition solution notebook was adapted. All numerical results derive from recorded executions or the Kaggle Submissions page.